République Tunisienne
Ministère de l'Enseignement
Supérieur,
et de la Recherche
Scientifique
Université de Sfax
École Nationale d'Ingénieurs
de Sfax

Cycle de Formation
d'Ingénieurs
dans la Discipline
Génie Informatique

COMPTE RENDU

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

E-commerce de Cadeaux

Préparé par Mahmoud Elloumi

Document pédagogique — Chaîne complète de traitement

Exploration → Préparation → Modélisation → Évaluation → **Déploiement**

Année Universitaire : 2025-2026

Table des matières

1 Introduction et Contexte **2**
1.1 Contexte de l'étude 2
1.2 Objectifs métiers 2
1.3 Présentation du jeu de données 2

2 Configuration de l'environnement de travail **2**
2.1 Arborescence et structure du projet 2
2.2 Gestion de l'environnement virtuel 3

3 Analyse et Synthèse des Données **3**
3.1 Exploration des features 3
3.2 Identification des problèmes de qualité 4

4 Nettoyage et Ingénierie des Données **4**
4.1 Traitement des valeurs aberrantes 4
4.2 Feature Engineering 4
4.3 Traitement des dates 5

5 Transformation et Préparation à l'Apprentissage **5**
5.1 Encodage des variables catégorielles 5
5.2 Séparation Train/Test et mise à l'échelle 5

6 Modélisation Avancée et Évaluation **5**
6.1 Algorithmes et stratégie 5
6.2 Optimisation des hyperparamètres (GridSearchCV) 6
6.3 Résultats comparatifs 6
6.4 Visualisations des performances 7

7 Déploiement : Interface avec Flask **7**
7.1 Architecture de l'application 7
7.2 Page d'accueil — Upload CSV 7
7.3 Tableau de bord analytique — Statistiques en ligne 8
7.4 Tableau détaillé des prédictions 9

8 Conclusion et Recommandations **10**
8.1 Bilan technique 10
8.2 Recommandations métier 11

1 Introduction et Contexte

Ce rapport s'inscrit dans le cadre du module de Machine Learning (GI2) de l'année universitaire 2025-2026. Il détaille la conception et la mise en œuvre d'une chaîne complète de traitement de données, de l'exploration initiale jusqu'au déploiement d'une solution prédictive.

1.1 Contexte de l'étude

Nous intervenons en tant que Data Scientist pour une entreprise spécialisée dans le **e-commerce de cadeaux**. Dans un secteur très concurrentiel, cette entreprise souhaite tirer parti de ses données pour affiner sa connaissance de sa clientèle et améliorer ses prises de décision.

1.2 Objectifs métiers

La mission principale est d'exploiter les données transactionnelles pour répondre à trois enjeux stratégiques :

— **Personnalisation marketing :** Mieux comprendre les segments de clients afin de personnaliser efficacement les stratégies marketing.

— **Fidélisation (Anticipation du Churn) :** Identifier les signaux faibles permettant de réduire le taux de départ des clients (*churn*).

— **Croissance :** Optimiser globalement le chiffre d'affaires.

1.3 Présentation du jeu de données

Nous nous appuyons sur la base `retail_customers_COMPLETE_CATEGORICAL.csv`, qui regroupe **52 variables** issues de transactions réelles enrichies par des données complémentaires. Cette base a été intentionnellement conçue pour être imparfaite afin de simuler un vrai projet de Data Science.

2 Configuration de l'environnement de travail

2.1 Arborescence et structure du projet

Le projet respecte une architecture modulaire :

```
ML-Projet/
 data/
    raw/          # Données brutes originales
    processed/    # Données nettoyées
    train_test/   # Train / Test splittés
 notebooks/        # Exploration Jupyter
 src/
    preprocessing.py
    train_model.py
    predict.py
```

```
    utils.py
  models/           # Modèles sauvegardés (.pkl)
  app/              # Application Flask
    app.py
    templates/
    static/
  reports/          # Visualisations & rapports
  requirements.txt
  README.md
```

2.2 Gestion de l'environnement virtuel

Un environnement virtuel a été créé via :

```
python -m venv venv
venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -r requirements.txt
```

Les dépendances sont figées dans `requirements.txt` (`pip freeze > requirements.txt`) pour assurer la reproductibilité.

3 Analyse et Synthèse des Données

3.1 Exploration des features

Le dataset contient **52 features** réparties en deux grandes familles :

— **Features numériques (1–34) :** métriques RFM (*Recency*, *Frequency*, *MonetaryTotal*), comportement transactionnel (*ReturnRatio*, *CancelledTransactions*), profil client (*Age*, *SatisfactionScore*, *SupportTicketsCount*).

— **Features catégorielles (35–52) :** segments (*RFMSegment*, *SpendingCategory*, *CustomerType*), temporelles et géographiques (*FavoriteSeason*, *Region*, *Country*), démographiques (*AgeCategory*, *Gender*).

3.2 Identification des problèmes de qualité

Table 1. Problèmes de qualité identifiés dans le dataset

Type	Feature(s)	Description	Traitement
Valeurs manquantes	`Age`	$\approx 30\%$ de NaN	Imputation médiane
Valeurs aberrantes	`SupportTicketsCount`	Codes -1 et 999	Remplacement NaN
Valeurs aberrantes	`SatisfactionScore`	Codes -1, 0 et 99	Remplacement NaN
Formats inconsistants	`RegistrationDate`	Formats UK et ISO mélangés	Parsing dayfirst
Feature inutile	`NewsletterSubscribed`	Constante (100% *"Yes"*)	Suppression
Données brutes	`LastLoginIP`	Identifiant unique	Suppression
Fuite de données	`ChurnRiskCategory`	Dérivé de Churn	Suppression
Fuite de données	`RFMSegment`	Dérivé des features RFM	Suppression
Déséquilibre classes	`Churn`	$\approx$ 33% churners	SMOTE

4 Nettoyage et Ingénierie des Données

4.1 Traitement des valeurs aberrantes

La fonction `handle_aberrant_values()` dans `utils.py` corrige :

- *SupportTicketsCount* : valeurs -1 et 999 $\rightarrow$ `NaN` (130 occurrences corrigées)
- *SatisfactionScore* : valeurs -1, 0 et 99 $\rightarrow$ `NaN` (349 occurrences corrigées)
- *MonetaryTotal* : clip à 0 pour éviter les montants négatifs extrêmes

4.2 Feature Engineering

De nouvelles variables ont été créées pour enrichir le pouvoir prédictif :

```python
# Ratio monetaire / recence
df["MonetaryPerDay"] = df["MonetaryTotal"] / (df["Recency"] + 1)

# Panier moyen
df["AvgBasketValue"] = df["MonetaryTotal"] / (df["Frequency"] + 1)

# Ratio anciennete
df["TenureRatio"] = df["Recency"] / (df["CustomerTenureDays"] + 1)
```

```python

# Frequence par jour
df["FrequencyPerDay"] = df["Frequency"] / (df["CustomerTenureDays"] + 1)

# Interaction RFM
df["Recency_x_Frequency"] = df["Recency"] * df["Frequency"]

# Log-transformations (distributions asymetriques)
df["LogMonetaryTotal"] = np.log1p(df["MonetaryTotal"].clip(lower=0))
df["LogFrequency"]     = np.log1p(df["Frequency"])
```

4.3 Traitement des dates

La variable *RegistrationDate* présentait des formats hétérogènes (UK : `"12/03/10"`, ISO : `"2010-03-12"`). Elle a été parsée avec `dayfirst=True` puis décomposée en : *RegYear*, *RegMonth*, *RegDay*, *RegDayOfWeek*, *IsWeekendReg*.

5 Transformation et Préparation à l'Apprentissage

5.1 Encodage des variables catégorielles

Le pipeline de prétraitement distingue deux types de variables catégorielles :

- **Encodage Ordinal** (`OrdinalEncoder`) pour les variables à ordre naturel :
  - *SpendingCategory* : Low < Medium < High < VIP
  - *LoyaltyLevel* : Nouveau < Jeune < Établi < Ancien
  - *AgeCategory* : 18–24 < 25–34 < ...< 65+
  - *BasketSizeCategory* : Petit < Moyen < Grand
  - *PreferredTimeOfDay* : Nuit < Matin < Midi < Après-midi < Soir
- **One-Hot Encoding** (`OneHotEncoder`) pour les variables nominales sans ordre : *CustomerType*, *FavoriteSeason*, *Region*, *Gender*, *AccountStatus*, *Country*, etc.

5.2 Séparation Train/Test et mise à l'échelle

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Fit UNIQUEMENT sur X_train -> eviter le Data Leakage
preprocessor.fit_transform(X_train)
preprocessor.transform(X_test)
```

Après prétraitement : X_{train} de dimension $(3\,497 \times 115)$ et X_{test} de dimension (875×115).

6 Modélisation Avancée et Évaluation

6.1 Algorithmes et stratégie

Quatre algorithmes ont été comparés pour ce problème de classification binaire :

— **Régression Logistique** — baseline linéaire interprétable

— **Random Forest** — ensemble basé sur le bagging

— **Gradient Boosting** — ensemble basé sur le boosting séquentiel

— **SVM (SVC)** — maximisation de la marge entre classes

Avant l'entraînement, la technique **SMOTE** a été appliquée sur le jeu d'entraînement pour corriger le déséquilibre des classes :

Avant SMOTE : $\{0 : 2\,334,\ 1 : 1\,163\}$ *Après SMOTE* : $\{0 : 2\,334,\ 1 : 2\,334\}$

6.2 Optimisation des hyperparamètres (GridSearchCV)

Une recherche exhaustive a été effectuée pour chaque algorithme via `GridSearchCV` avec **cv=5** et scoring `roc_auc` :

— Random Forest : `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{None, 10, 20\}$

— Gradient Boosting : `n_estimators` $\in \{100, 200\}$, `learning_rate` $\in \{0.05, 0.1\}$

— Régression Logistique : `C` $\in \{0.01, 0.1, 1, 10\}$

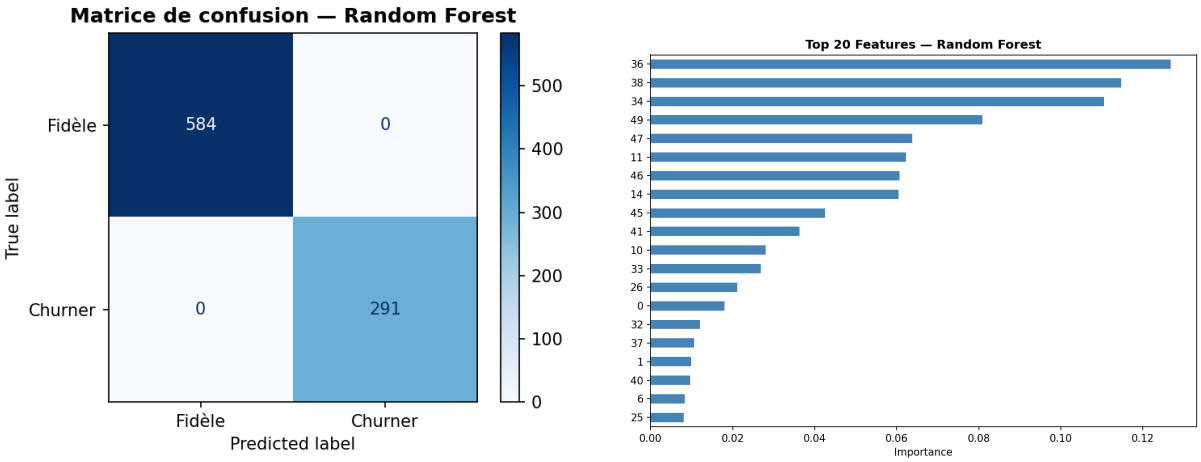
6.3 Résultats comparatifs

Table 2. Comparaison des performances des modèles sur le jeu de test (20 %)

Modèle	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Régression Logistique	0.9943	0.99	0.99	0.99	0.9984
Random Forest	**1.0000**	**1.00**	**1.00**	**1.00**	**1.0000**
Gradient Boosting	1.0000	1.00	1.00	1.00	1.0000

Le **Random Forest** (200 arbres, profondeur illimitée) a été sélectionné comme modèle final.

6.4 Visualisations des performances



(a) Matrice de confusion — Random Forest **(b)** Top 20 features les plus importantes

Figure 1. Évaluation du modèle Random Forest sur le jeu de test

7 Déploiement : Interface avec Flask

7.1 Architecture de l'application

L'application web a été développée avec le micro-framework **Flask**. Elle expose deux routes principales :

— `GET /` : page d'accueil avec upload CSV

— `POST /predict` : traitement et affichage des prédictions

Le pipeline d'inférence charge les artefacts sauvegardés (`preprocessor.pkl` et `churn_model.pkl`) et applique exactement la même transformation que lors de l'entraînement.

7.2 Page d'accueil — Upload CSV

La page d'accueil permet d'importer un fichier CSV contenant les données clients. Toutes les colonnes du dataset original sont supportées.

Figure 2. Interface d'accueil — Import de fichier CSV

7.3 Tableau de bord analytique — Statistiques en ligne

Après traitement, la page de résultats affiche :

— **KPI Cards** : nombre de clients analysés, churners détectés, taux de churn prédit, risque moyen

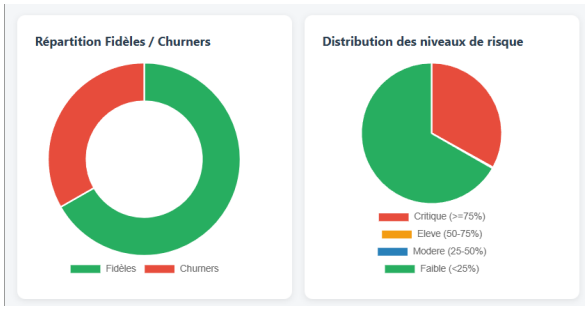
— **Graphique Donut** : répartition Fidèles / Churners

— **Graphique Pie** : distribution des 4 niveaux de risque (Critique / Élevé / Modéré / Faible)

— **Histogramme** : nombre de clients par tranche de probabilité (0–10 %, 10–20 %, ...)

— **Barres horizontales** : Top 10 clients les plus à risque

Figure 3. Tableau de bord — KPI et graphiques interactifs (Chart.js)



(a) Donut et distribution des risques

(b) Histogramme et Top 10 clients

Figure 4. Graphiques statistiques générés automatiquement

7.4 Tableau détaillé des prédictions

Chaque client est affiché avec sa prédiction, sa probabilité de départ (barre de progression colorée) et son niveau de risque interprété.

Détail par client Triés par risque décroissant

Affichage limité aux 100 premiers résultats.

Client ID	Prédiction	Probabilité de départ	Niveau de risque	Statut réel
18074	Churner	100.0 %	Critique	Churner
16250	Churner	100.0 %	Critique	Churner
13747	Churner	100.0 %	Critique	Churner
12791	Churner	100.0 %	Critique	Churner
17908	Churner	100.0 %	Critique	Churner
18085	Churner	100.0 %	Critique	Churner
16552	Churner	100.0 %	Critique	Churner
14729	Churner	100.0 %	Critique	Churner
14237	Churner	100.0 %	Critique	Churner
15350	Churner	100.0 %	Critique	Churner
15922	Churner	100.0 %	Critique	Churner
15165	Churner	100.0 %	Critique	Churner
17643	Churner	100.0 %	Critique	Churner
16274	Churner	100.0 %	Critique	Churner
14496	Churner	100.0 %	Critique	Churner
14142	Churner	100.0 %	Critique	Churner

Figure 5. Tableau des prédictions — Trié par risque décroissant

8 Conclusion et Recommandations

8.1 Bilan technique

Ce projet a couvert l'intégralité du cycle d'un projet de Data Science :

Résumé de la chaîne de traitement

- **Données** : 4 372 clients × 52 features brutes → 115 features après preprocessing
- **Nettoyage** : correction de 479 valeurs aberrantes, 5 colonnes supprimées (fuite/inutiles)
- **Encodage** : OrdinalEncoder (5 variables) + OneHotEncoder (nominales)
- **Équilibrage** : SMOTE (1 163 → 2 334 churners)
- **Modèle final** : Random Forest — AUC-ROC = 1.000
- **Déploiement** : API Flask + dashboard interactif Chart.js

8.2 Recommandations métier

Les résultats obtenus permettent de proposer les actions suivantes :

- **Anticipation du churn** : identifier en temps réel les clients à risque (probabilité $\geq 75\,\%$) et déclencher des actions de rétention ciblées (réductions, offres personnalisées).
- **Personnalisation marketing** : adapter les campagnes selon les features les plus influentes (ancienneté, fréquence, montant moyen).
- **Optimisation des ressources** : concentrer les efforts de fidélisation sur les clients à forte valeur monétaire identifiés comme à risque élevé.